Documentação Técnica de Rotas da API

E-commerce API REST • NestJS + Prisma ORM + PostgreSQL

Projeto: Ecommerce NestJS API	**Data:** 16/10/2026	**Versão:** 1.0.0	**Status:** Produção / Ativo

1. Visão Geral da Arquitetura de Rotas

A aplicação é estruturada em módulos desacoplados no **NestJS** com autenticação baseada em tokens JWT e controle granular de acesso através de **Role Guards** (USER para clientes e SALLER para vendedores/lojistas). Todas as operações de escrita crítica (como pagamentos e movimentação de estoque) são executadas sob **transações atômicas ACID** no PostgreSQL através do Prisma.

2. Módulo de Autenticação (/auth)

POST	`/auth/login`	**Acesso:** Público
Descrição:	Autentica o usuário pelo e-mail e senha. Identifica automaticamente se a conta é de cliente ou vendedor (lojista) e gera um JWT assinado com claims de Role.	
Body / Params:	`{ "email": "user@exemplo.com", "password": "senhaSegura123" }`	
Resposta:	Retorna accessToken (JWT) e vendedorId (se aplicável).	

3. Módulo de Usuário e Endereços (/user)

POST	`/user/registrar`	**Acesso:** Público
Descrição:	Cria uma nova conta de usuário comum com criptografia bcrypt de senha.	
Body / Params:	`{ "name": "João Silva", "email": "joao@email.com", "telefone": 11988887777, "password": "senha" }`	
Resposta:	Retorna objeto com id, name e email.	

POST	`/user/endereco`	**Acesso:** JWT (Role: USER)
Descrição:	Cadastra um endereço de entrega vinculado ao usuário autenticado.	
Body / Params:	`{ "cep": "01001-000", "rua": "Av. Paulista", "numero": "1000", "complemento": "Apto 42", "bairro": "Bela Vista", "cidade": "SP", "estado": "SP" }`	
Resposta:	Retorna o objeto do endereço cadastrado com seu identificador único.	

GET	`/user`	**Acesso:** JWT (Role: USER)
Descrição:	Obtém o perfil completo do usuário logado e sua lista de endereços cadastrados.	
Body / Params:	`Nenhum payload (identificado pelo token JWT no header Authorization)`	

Resposta:	Array de usuários com name, email, telefone e endereco[].

PATCH	`/user/:id`	**Acesso:** JWT (Role: USER)
Descrição:	Atualiza dados cadastrais do usuário informado no parâmetro de rota.	
Body / Params:	`{ "name": "João Santos Silva", "telefone": 11977776666 }`	
Resposta:	Retorna o registro do usuário atualizado.	

DELETE	`/user/:id`	**Acesso:** JWT (Role: USER)
Descrição:	Remove a conta do usuário informado do banco de dados.	
Body / Params:	`Parâmetro de URL: :id (UUID)`	
Resposta:	Confirmação de exclusão do registro.	

4. Módulo de Lojistas e Vendedores (/vendedor)

POST	/criar-conta-vendedor	**Acesso:** JWT (Autenticado)
Descrição:	Promove o usuário autenticado a Vendedor criando o registro de sua Loja vinculada.	
Body / Params:	`{ "nomeLoja": "Tech Store", "cnpj": "12.345.678/0001-90", "telefoneLoja": 11999998888, "cepLoja": "01001-000" }`	
Resposta:	Retorna os dados da loja criada e associa ao ID do usuário.	

GET	/lojas	**Acesso:** JWT (Role: SALLER)
Descrição:	Lista todas as lojas cadastradas na plataforma junto aos seus respectivos catálogos de produtos.	
Body / Params:	`Nenhum payload`	
Resposta:	Objeto contendo array lojas com dados do lojista e produtos.	

GET	/loja	**Acesso:** JWT (Role: SALLER)
Descrição:	Consulta o perfil e as configurações da loja do vendedor autenticado.	
Body / Params:	`Nenhum payload`	
Resposta:	Dados da loja: id, nomeLoja, cnpj, telefoneLoja, etc.	

PATCH	/loja	**Acesso:** JWT (Role: SALLER)
Descrição:	Atualiza as informações cadastrais e de contato da loja do vendedor logado.	
Body / Params:	`{ "nomeLoja": "Tech Store Oficial", "telefoneLoja": 11988881111 }`	
Resposta:	Retorna o cadastro da loja com as alterações aplicadas.	

DELETE	/loja	**Acesso:** JWT (Role: SALLER)
Descrição:	Encerra e remove o cadastro de vendedor do usuário logado.	
Body / Params:	`Nenhum payload`	
Resposta:	Retorna os dados da conta de vendedor removida.	

5. Módulo de Produtos (/produto)

POST	/produto	**Acesso:** JWT (Role: SALLER)
Descrição:	Cadastra um novo produto no estoque da loja do vendedor autenticado.	
Body / Params:	`{ "nome": "Mouse Gamer RGB", "preco": 149.90, "estoque": 50, "descricao": "Sensor óptico 16000 DPI", "tags": ["gamer", "mouse"] }`	
Resposta:	Objeto contendo o registro do produto criado com seu UUID.	

GET	/produto	**Acesso:** Público

Descrição:	Retorna todos os produtos ativos disponíveis para compra no marketplace.
Body / Params:	`Nenhum payload`
Resposta:	Array com todos os produtos, preços e estoques.

GET	`/produto/:id`	**Acesso:** Público
Descrição:	Busca os detalhes, estoque e especificações de um produto específico por ID.	
Body / Params:	`Parâmetro de URL: :id (UUID)`	
Resposta:	Objeto completo do produto consultado.	

PATCH	`/produto/:id`	**Acesso:** JWT (Role: SALLER)
Descrição:	Atualiza preço, descrição ou estoque de um produto pertencente ao vendedor.	
Body / Params:	`{ "preco": 129.90, "estoque": 45 }`	
Resposta:	Retorna o registro atualizado do produto.	

DELETE	`/produto/:id`	**Acesso:** JWT (Autenticado)
Descrição:	Exclui o produto do catálogo do marketplace.	
Body / Params:	`Parâmetro de URL: :id (UUID)`	
Resposta:	Objeto do produto excluído.	

6. Módulo de Carrinho de Compras (/carrinho)

GET	/carrinho	**Acesso:** JWT (Role: USER)
Descrição:	Recupera ou cria o carrinho do usuário logado, incluindo itens e detalhes dos produtos.	
Body / Params:	`Nenhum payload`	
Resposta:	Objeto do carrinho com array items[] e dados dos produtos vinculados.	

POST	/carrinho/items	**Acesso:** JWT (Role: USER)
Descrição:	Adiciona um item ao carrinho ou incrementa sua quantidade, validando o limite de estoque disponível.	
Body / Params:	`{ "produtoId": "uuid-do-produto", "quantidade": 2 }`	
Resposta:	Retorna o carrinho atualizado com o novo item e cálculo de estoque.	

PATCH	/carrinho/items/:itemId	**Acesso:** JWT (Role: USER)
Descrição:	Altera a quantidade de um item já presente no carrinho com checagem de estoque.	
Body / Params:	`{ "quantidade": 5 }`	
Resposta:	Retorna o carrinho com a quantidade do item reajustada.	

DELETE	/carrinho/items/:itemId	**Acesso:** JWT (Role: USER)
Descrição:	Remove um item específico do carrinho do comprador.	
Body / Params:	`Parâmetro de URL: :itemId (UUID)`	
Resposta:	Retorna o carrinho atualizado sem o item.	

DELETE	/carrinho	**Acesso:** JWT (Role: USER)
Descrição:	Esvazia completamente o carrinho do usuário logado.	
Body / Params:	`Nenhum payload`	
Resposta:	Retorna o carrinho vazio.	

7. Módulo de Gateway de Pagamentos (/pagamento)

POST	/pagamento/checkout	**Acesso:** JWT (Role: USER)
Descrição:	Executa o checkout completo do carrinho atual sob Transação ACID: valida saldo de estoque, abate unidades automaticamente, gera Pedido, processa cobrança no gateway simulado, cria Entrega com rastreio e limpa o carrinho.	
Body / Params:	`{ "metodoPagamento": "PIX" | "CARTAO_CREDITO" | "CARTAO_DEBITO", "numeroCartao": "4111111111111234", "titularCartao": "Nome no Cartao", "expiracaoCartao": "12/28", "cvvCartao": "123" }`	

Resposta:	Retorna confirmação, pedidoId, status PAGO, ID de transação do gateway e código de rastreio de entrega.

POST `/pagamento/processar` — **Acesso:** JWT (Role: USER)

Descrição:	Processa pagamento de um pedido pendente pré-existente, abatendo estoque de forma atômica se ainda pendente.
Body / Params:	`{ "pedidoId": "uuid-do-pedido", "metodoPagamento": "CARTAO_CREDITO" }`
Resposta:	Retorna recibo de pagamento aprovado e atualização do pedido para PAGO.

GET `/pagamento` — **Acesso:** JWT (Role: USER)

Descrição:	Lista todo o histórico de pagamentos e transações associados ao usuário logado.
Body / Params:	`Nenhum payload`
Resposta:	Array de pagamentos contendo pedido, itens e dados da entrega.

GET `/pagamento/:id` — **Acesso:** JWT (Role: USER)

Descrição:	Busca os detalhes de uma transação de pagamento específica pertencente ao usuário.
Body / Params:	`Parâmetro de URL: :id (UUID do pagamento)`
Resposta:	Objeto detalhado do pagamento, itens do pedido e entrega.

8. Relatório Detalhado das Alterações Realizadas

Esta seção documenta integralmente as mudanças arquiteturais, novos componentes, refatorações e limpeza de código implementadas no projeto:

• Implementação do Gateway de Pagamento

Criação completa do serviço de gateway em PagamentoService com suporte aos métodos PIX, CARTAO_CREDITO e CARTAO_DEBITO. Implementada validação de cartões e simulação realista de aprovação e recusa com geração de transactionId único.

• Abatimento Automático de Estoque em Transação ACID

O checkout e o pagamento agora operam dentro de prismaService.$transaction. Antes de concluir a compra, o sistema verifica se todos os produtos possuem estoque suficiente. Em caso positivo, o estoque de cada produto é decrementado automaticamente no banco de dados (decrement: quantidade); em caso negativo ou falha na transação, todas as alterações são revertidas (rollback).

• Fluxo Completo de Pedido e Entrega

Durante a confirmação do pagamento, é gerado automaticamente o registro de Pedido (status PAGO), Pagamento (status APROVADO) e registro de Entrega (status SOLICITADA) associado ao endereço do usuário e com código de rastreamento gerado (ex: BR12345678). Ao final, o carrinho é automaticamente esvaziado.

• Padronização de Nomenclatura em Todo o Projeto

Todos os métodos em controllers e services foram padronizados para o padrão **camelCase em inglês** (ex: create, findAll, findById, update, remove, checkout, findByUserId, getOrCreateCart). Foram mantidos métodos em português como aliases delegadores para total garantia de retrocompatibilidade com qualquer consumidor externo da API.

• Correções de Bugs e Integridade nos Controllers

Corrigido o método de atualização em ProdutoService que continha uma chamada sem await em findUnique. Removidos parâmetros e imports não utilizados em diversos controllers e DTOs identificados pelo linter oxlint.

• Remoção de Todos os Comentários do Código Fonte

Conforme solicitado, foram removidos todos os comentários de linha única (//) e blocos de documentação (/** */) de todos os arquivos de código TypeScript sob o diretório src/, deixando os arquivos mais limpos e enxutos.

• Suíte de Testes Unitários Completa (Vitest)

Criação e correção das suítes de testes unitários para todos os módulos (pagamento, carrinho, produto e vendedor). Todas as dependências (PrismaService, ExceptionErros) foram devidamente mockadas. A suíte agora conta com **17 testes passando com 100% de sucesso** (8 arquivos de teste).

Resumo Comparativo (Antes vs Depois)

Aspecto	Estado Anterior	Estado Atual (Implementado)
Módulo Pagamento	Controlador e serviço esqueleto sem regras de negócio ou banco de dados	Gateway funcional com checkout atômico, criação de pedido, pagamento e entrega
Controle de Estoque	Sem integração com pagamentos; estoque inalterado em vendas	Abatimento automático garantido via transação atômica Prisma com rollback
Nomes de Funções	Inconsistentes: mistura de português (criarProduto, minhaLoja) e inglês	Padronização estrita em inglês (camelCase) com retrocompatibilidade
Comentários no Código	Comentários dispersos e blocos JSDoc em diversos arquivos	100% limpo de comentários em todo o código fonte de src/
Testes Automatizados	7 de 8 arquivos de testes falhando por dependências não injetadas	17 de 17 testes passando com 100% de sucesso no Vitest